
15-853 Project: Comparative Study of Nearest Neighbour Techniques

Abhinav Jauhri

Department of Electrical and Computer Engineering
Carnegie Mellon University
Pittsburgh, PA 15213
ajauhri@cmu.edu

1 Introduction

For this project we implemented two algorithms for finding nearest-neighbours(NN). NN algorithms are relevant for machine learning applications since it is believed that high-dimensional datasets have low-dimensional intrinsic structure. Specifically, we shall discuss performance and accuracy results of cover trees [2], and locality sensitive hashing [3] on different datasets and perform a comparative study.

We will first provide an informal description of *Cover Trees* and *Locality Sensitive Hashing* algorithm. For a formal description, refer to [2, 3]. In the rest of the paper, we shall use training data set and sample data set interchangeably.

1.1 Cover Trees

A cover tree is a levelled tree comprising of a collection of data points S in some n -dimensional space. Each level is represented by a scale i which decreases as we move away from the root element. Please note that cover trees work on arbitrary metrics but bounds depend on expansion constant c which defines worst ratio between the number of points in the parent cover set and child cover set. A cover set is denoted by C_i in Figure 1.

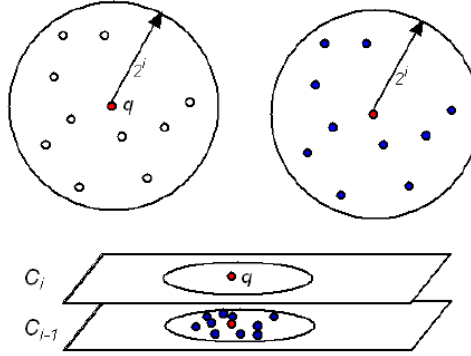


Figure 1: Covering set (Image credits: Victoria Choi)

There are two important invariants which are essential to guarantee an efficient running time of the algorithm:

- For every element $p \in C_{i-1}$, there exists a parent $q \in C_i$ such that $d(q, p) \leq 2^i$
- For all distinct points $p, q \in C_i$, $d(p, q) > 2^i$

Based on the above invariants, one can derive bounds on the number of children a node can have, and more importantly a bound on the depth. For bounds in Table 1, n is the number of data points in S .

Table 1: Bounds on Cover Trees

Parameter	Bound
depth	c^4
search	$O(c^{12} \log n)$
insertion	$O(c^6 \log n)$
deletion	$O(c^6 \log n)$

1.2 Locality Sensitive Hashing (E^2 LSH)

The LSH algorithm used here are formed using p -stable distributions specifically *Gaussian distribution*. The basic intuition behind using such a hash function is this: if a were a d dimensional vector with entries chosen independently from a p -stable distribution, then the dot product with some vector v i.e. $a \cdot v$ will project the vector to some real line. Further, the distance between two vectors after projection $a \cdot v_1 - a \cdot v_2$ is distributed as $\|v_1 - v_2\|_p \cdot X$, where X is a p -stable distribution. Hence, such projections are locality preserving such that points which lie close to others after projection, are most likely to be close in the d dimensional space as well [1].

2 Implementation

Both algorithms were implemented in Python 2.7.5+.

2.1 Cover Trees

Cover trees have an object-oriented data structure. Each node in the tree is represent by objects of class `tree_node`:

```
class tree_node:
    def __init__(self, point):
        self.point = point
        self.max_scale = 0
        self.min_scale = 0
        self.children = {}
```

References to children of a node entry are stored as a dictionary using `scale` as the key entry of all children. Currently the packages implements the following functions:

- `create(X)` - To create a cover tree where X is a matrix of elements in some n -dimensional space.
- `insert(p, root, max_scale)` - To add an element to the cover tree defined by `root`. `root` is returned by `create(...)`.
- `knn(knn, p, root)` - To find the k nearest neighbours for a given point p

2.2 E^2 LSH

Memory utilization was the major bottleneck in the implementation of the hashing scheme. For this reason, we implemented *hybrid-type* data structure described in [1]¹ such that it requires $O(nL)$ memory, where n is the number of elements for each hash function and L is the number of functions such that each function is a set of hash h_i functions chosen uniformly at random from the LSH family H . The constant for $O(nL)$ is 12 bytes as shown in class `hybird_entry` below.

```
class hybird_entry(C.Structure):
    _fields_ = [("is_last_bucket", C.c_uint),
                ("bucket_length", C.c_uint),
                ("is_last_point", C.c_uint),
```

¹For detailed description please refer to Section 3.6 of the cited reference

Table 2: Number of children and number of nodes for Netflix data

# of children	# of Node(s)
0	1319
1	336
2	118
3	70
4	30
5 - 48	83
345	1

```

("point_index", C.c_uint)]

class hybrid_chain_entry(C.Union):
    _fields_ = [("control_value", C.c_uint),
                ("point", hybrid_entry),
                ("is_null", C.c_int)]

```

By *hybrid-type*, the storage refers to the fact that each of the L functions have a table which stores both buckets' and points' description. Therefore, the length of such a table is $\#buckets + n$. The E²LSH package exposes the following functionalities:

- `start(X, r)` - To create the hash tables using training data in X . r is the upper bound on the radius for which nearest neighbours will be returned.
- `get_ngh_struct(nn, q)` - To find the nearest neighbours for an element q within a radius r . The entire set of hash functions are defined by nn which is returned by `start(...)`

3 Experiments

Timings for the implementation are from experiments executed on 8-CPU Intel Ivy Bridge processor. The datasets in experiments include a sparse matrix of movie reviews from Netflix ($480,189 \times 3004$)² and handwritten digit database ($60,000 \times 784$)³.

3.1 Timings

Cover trees are known to not perform optimally in terms of querying time given a certain structure of data. More precisely, this is the case when the expansion constant gets too large. For intuition imagine if a ball of radius r covers a certain set of points, then how many balls of radius r will be required to cover points which are $(r + \text{constant})$ away in a high dimensional space [4]? The number of balls required is directly proportional to the expansion constant. Therefore, we observe that cover trees do not scale well with sparse datasets. This is validated in the observations in Table 2 where one node had 345 children out of a sample size of $2K$. Presence of such clusters is possible since you may expect a group of movie reviewers to have similar interests. Due to this reason the performance of our insertions to cover trees for sparse data degraded, and we performed experiments on sparse data for sample size $\leq 20K$ entries.

For E²LSH we pre-computed the optimal values for two constants - L and k . k is the number of hash functions in each g_i . This was achieved by logging timings for two operations - 1) T_g is the total time to compute L functions g_i for a query point; 2) T_c is the time for computing distances to all points found in the same bucket as the query point. It should be noted that T_g and T_c are not correlated. `compute_opt` function in the source code at high level aims to minimize the mean query time. The timing results here do not include the cost of pre-computing the constants.

²Rows represent users and columns represent movies. Eventually we used a subset of this dataset.

³<http://yann.lecun.com/exdb/mnist/>

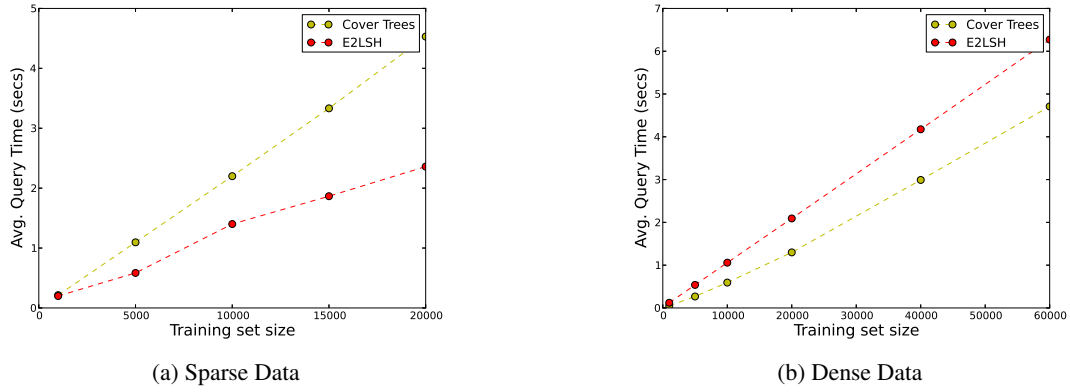


Figure 2: Average query was taken over 10 random points for 5 NNs

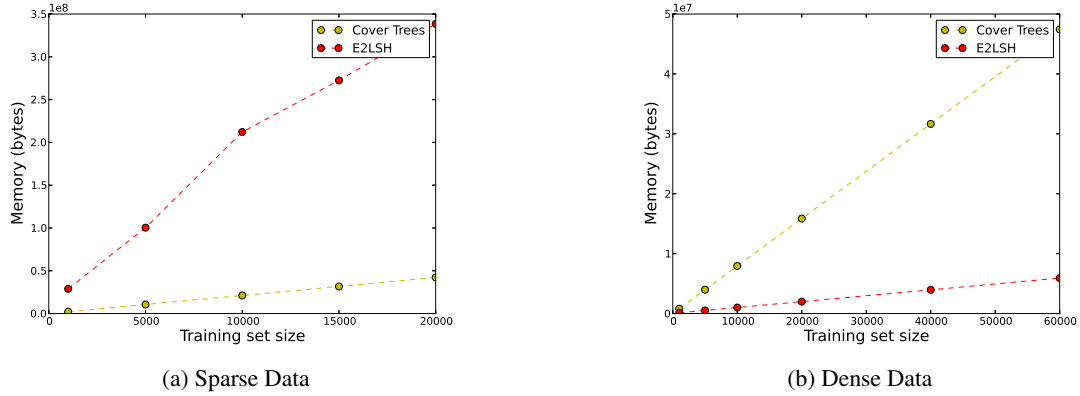


Figure 3: Memory utilization for NN algorithms

3.2 Memory

Figure 3 provides the memory utilization for creating data structures for the two NN algorithms. For cover trees, we include the memory used by the construction of the tree using a particular sample size. Similarly, for E²LSH we include the construction cost of the data structure which includes all L sets of hashing functions. The disparity in the memory utilization for E²LSH for sparse and dense datasets is mainly due to the large value of $L = 595$ for sparse data as compared to $L = 10$ for dense data obtained by the `compute_opt` function.

4 Image Patching

Finally, we also experimented E²LSH to find good image patches for an incomplete image. We used SIFT [5] to extract image features from a patch of an image. Consider Figure 4 : unique patches of 12×12 pixels were created with *bin size* = 3. Unique because the *sampling step* for the subsequent patch was 12 hence ensuring there to be no overlapping between two consecutive patches. The following algorithm was used to find approximate patches of an incomplete image:

- From the original image I , black out some n equal sized patches. Call the resultant image T .
- Apply SIFT on T to get features and mid-points of all patches whether blackened or not.
- Apply E²LSH on T

- Find 4 neighbours for each of the n patches in each of the directions i.e. top, bottom, right, and left. Refer them to as $t, b, r,$ & l respectively.
- Query the neighbour hood structure returned by the hashing algorithm with $4 \times n$ points. Call the set of LSH neighbours as N .
- Use the closest neighbour for each $t, b, r,$ & $l \in N_i, N_{i+1}, N_{i+2}, N_{i+3}$ respectively to find patches which lie in the same direction as the blackened patch relative to $t, b, r,$ or l . Call this set of patches R
- Apply the patch $\in R$ to the blackened patch which closest to its corresponding $t, b, r,$ or l

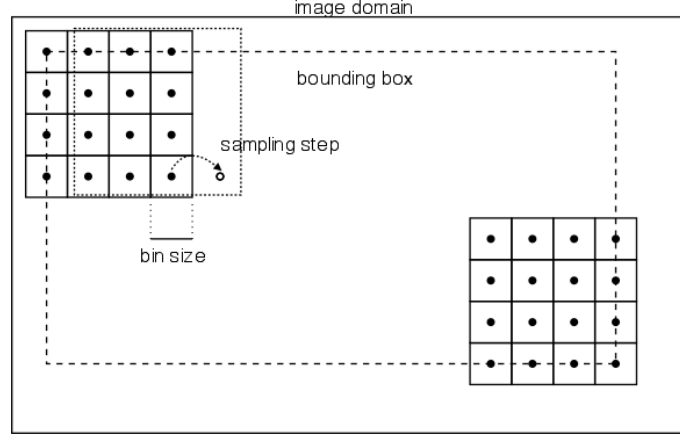


Figure 4: Image Patching (Image source: VLFeat.org)

The resultant image with almost near neighbours is shown on the left of Figure 6. Intuitively, these could improve with having trained the LSH algorithm with a larger corpus of data. Specifically, capturing different frames of an image with respect to time. In terms of RMSE, we observe a linear rise with increase in the number of blackened patches (Figure 5).

REFERENCES

- [1] Alexandr Andoni and Piotr Indyk. E2lsh user manual. 2005.
- [2] Alina Beygelzimer, Sham Kakade, and John Langford. Cover trees for nearest neighbor. In *Proceedings of the 23rd international conference on Machine learning*, pages 97–104. ACM, 2006.
- [3] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S Mirrokni. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the twentieth annual symposium on Computational geometry*, pages 253–262. ACM, 2004.
- [4] Robert Krauthgamer and James R Lee. Navigating nets: simple algorithms for proximity search. In *Proceedings of the fifteenth annual ACM-SIAM symposium on Discrete algorithms*, pages 798–807. Society for Industrial and Applied Mathematics, 2004.
- [5] David G Lowe. Object recognition from local scale-invariant features. In *Computer vision, 1999. The proceedings of the seventh IEEE international conference on*, volume 2, pages 1150–1157. Ieee, 1999.

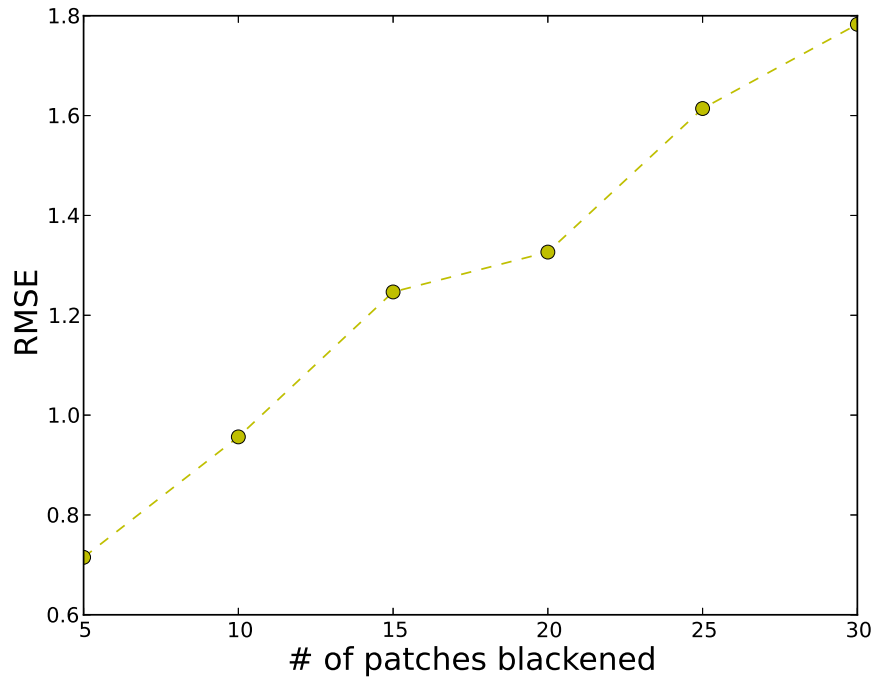


Figure 5: RMSE for image patching using E^2 LSH

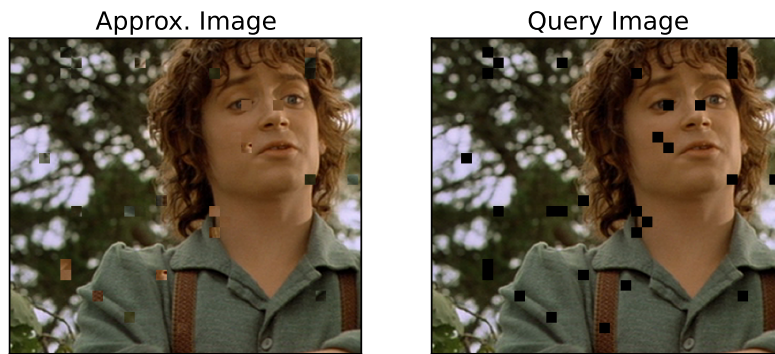


Figure 6: Image patching using E^2 LSH with query image of 30 patches each of size 12×12 pixels